Module 3

Coverage Planning & Analysis in Practice (SV/UVM)

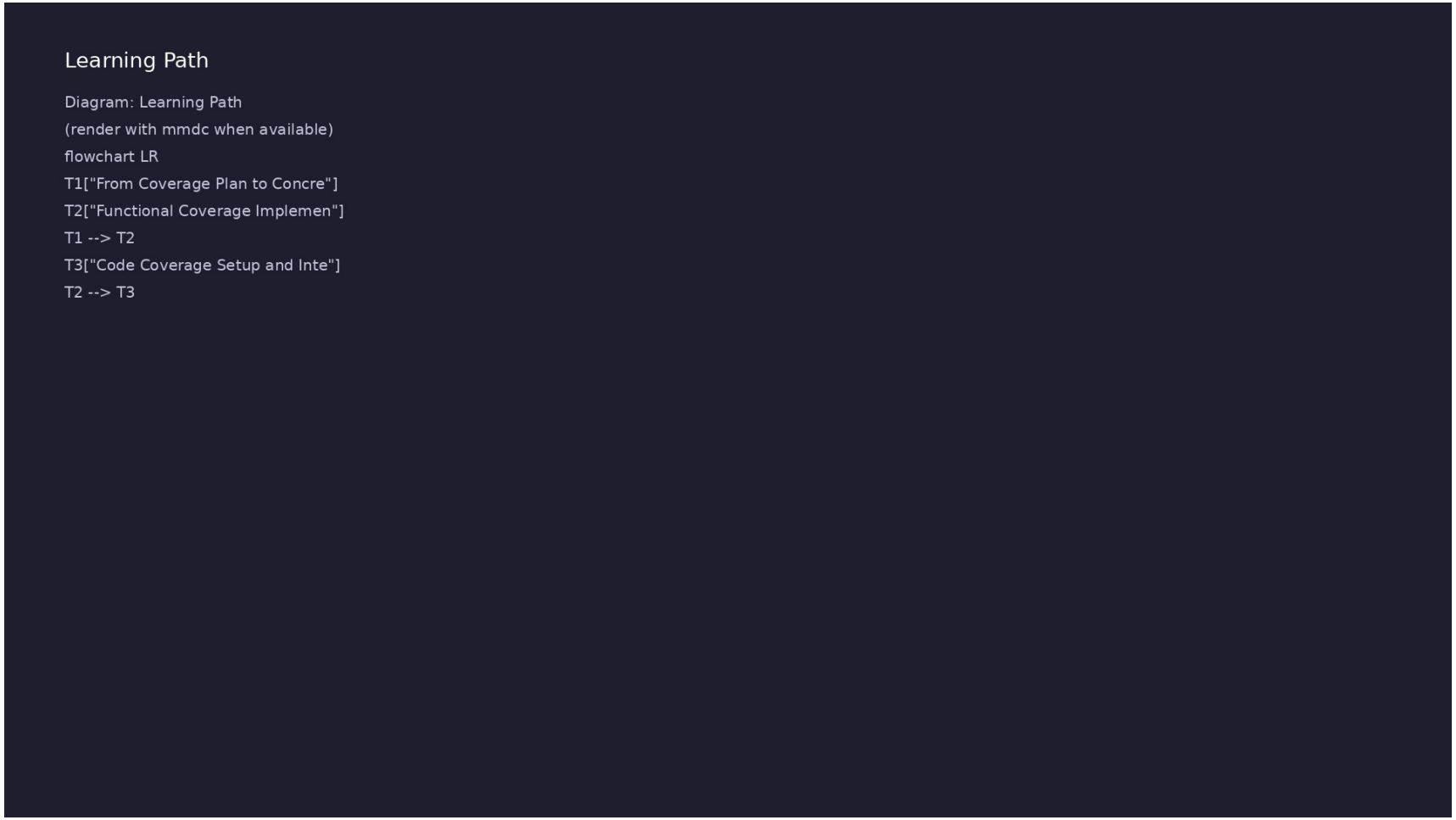
Learning objectives

- Design and implement a practical **coverage model**
  (functional + code), connect it to your UVM env…

Prerequisites

- See module README

Learning path



Design and implement a practical **coverage model** (functional + code), connect it to your UVM environment and test plan, and perform firs…

Overview

- Modules 1–2 focused on **what to test** and **how to structure tests and regressions**. Module 3 tu…

How to learn this module

Suggested learning path (1/2)

- Re-open `module2/TEST_PLAN.md` and `module2/COVERAGE_PLAN.md`.
- Scaffold Module 3 workspace: `./scripts/module3.sh --scaffold`
- Design covergroups in `module3/COVERAGE_DESIGN.md` and plan runs in `COVERAGE_RUN.md`.
- Implement or review `common_dut/tb/stream_fifo_coverage.sv` covergroup definitions.
- Run a small CORE-tier regression and capture coverage (document in `COVERAGE_RUN.md`).

Suggested learning path (2/2)

- Validate: `./scripts/module3.sh --check`

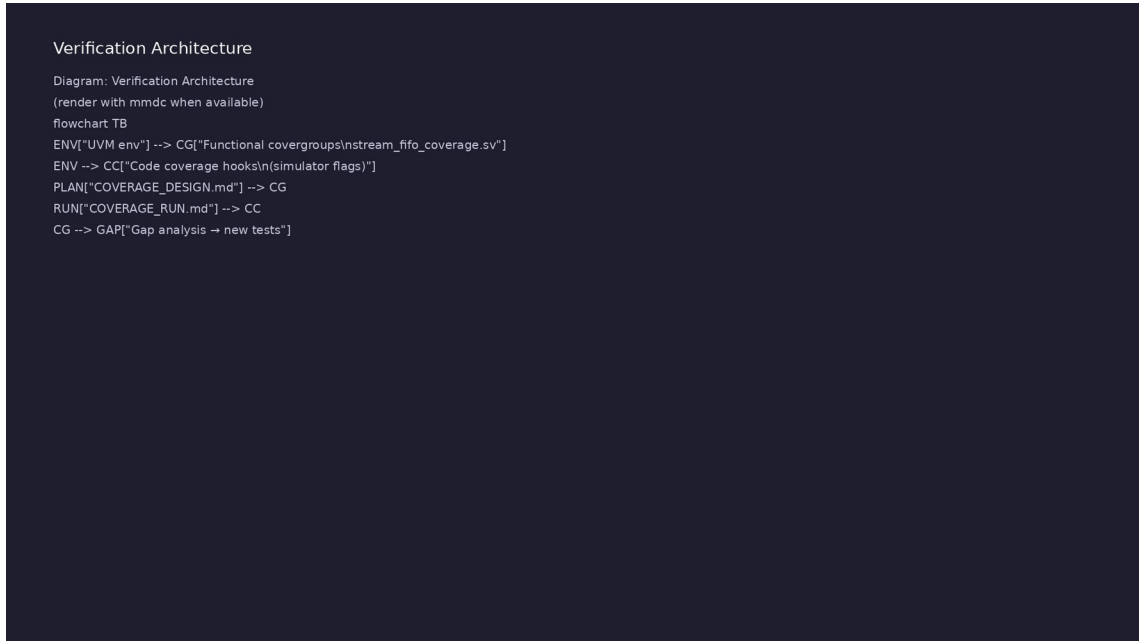
Design architecture

1. Coverage in the UVM environment

- Functional coverage lives in `common_dut/tb/stream_fifo_coverage.sv` (covergroups, crosses).
- Monitors/agents sample transactions that feed coverpoints (fill level, handshake patterns, errors).
- `COVERAGE_DESIGN.md` is the authoritative map from requirements → coverpoints → tests.

2. Code coverage integration

- Simulator line/branch/FSM coverage configured in your run scripts (documented in `COVERAGE_RUN.md`).
- Separate functional vs code coverage roles: intent vs implementation exercised.



3. Coverage run and merge pipeline

- Run: select CORE-tier tests → enable functional + code coverage in simulator → save per-test databa…
- Merge: combine run databases into a session or regression snapshot for analysis.
- Analyze: compare merged coverage against goals in `COVERAGE_PLAN.md`; record gaps in `COVERAGE_RUN.…
- Close: add targeted tests or refine coverpoints; re-run until must-cover bins are hit.

Coverage model — covergroups

```
covergroup cg_ops;
    option.per_instance = 1;
    cp_op_type: coverpoint op_type {
      bins idle          = { 0 };
      bins push_only     = { 1 };
      bins pop_only      = { 2 };
      bins push_and_pop  = { 3 };
    }
  endgroup

  //
------------------------------------------------------------------
------
  // CG_LEVEL: Occupancy levels (empty / low / mid / high / full)
  //
------------------------------------------------------------------
------
  covergroup cg_level;
```

Key files to study

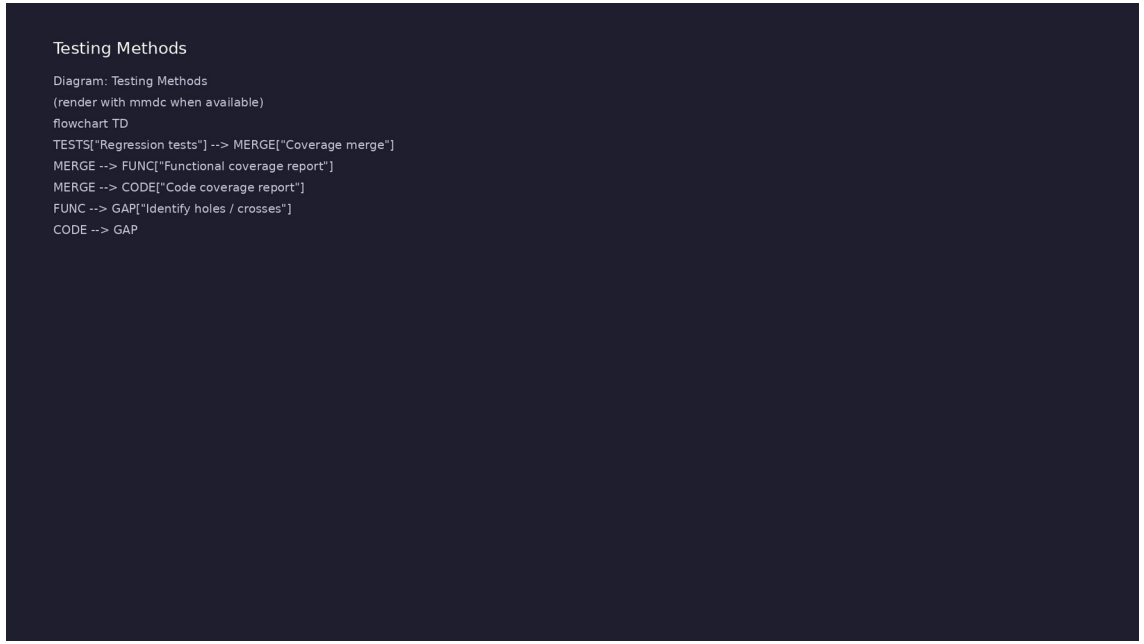
Open these in the repo

- `module3/COVERAGE_DESIGN.md` — coverpoints, crosses, and requirement mapping
- `module3/COVERAGE_RUN.md` — run logs, merge steps, and gap analysis notes
- `common_dut/tb/stream_fifo_coverage.sv` — functional coverage implementation
- `module2/COVERAGE_PLAN.md` — coverage goals and tier linkage
- `scripts/module3.sh` — coverage design and TB presence checks

Verification & testing methods

1. Coverage-driven verification (CDV)

- Run a CORE-tier regression subset, merge coverage, and compare against goals in `COVERAGE_PLAN.md`.
- Use gap analysis to add targeted tests or relax/remove low-value bins — not blind random runs.



2. Closure loop

- Requirements → tests → coverpoints → measured bins → updated plan/tests.
- `./scripts/module3.sh --check` expects covergroup references and coverage design docs.

3. Reporting discipline

- Log each run in `COVERAGE_RUN.md` with tool, seed, tier, and top uncovered items.

sample() integration point

```
function new(int depth_param = 16);
    depth = depth_param;
    cg_ops   = new();
    cg_level = new();
    cg_flags = new();
  endfunction

  // Call from monitor on posedge clk after driving level, op_type,
overflow, underflow
  function void sample();
    cg_ops.sample();
    cg_level.sample();
    cg_flags.sample();
  endfunction
```

Module 3 — coverage checks

```
#!/usr/bin/env bash

# Module 3: Coverage Planning & Analysis Orchestrator
# This script summarizes and checks the coverage artifacts for Module
3.
# Usage: ./scripts/module3.sh [OPTIONS]

set -euo pipefail

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
```

Syllabus topics

1. From Coverage Plan to Concrete Model

- Refinement of Coverage Goals
- Translating high-level goals into specific covergroups and bins.
- Deciding which metrics are *must-hit* vs *nice-to-have*.
- Coverage Model Structure
- Organizing covergroups by feature, interface, or component.
- Choosing the right placement (agent monitor, scoreboard, reference model, etc.).

2. Functional Coverage Implementation (SV/UVM) (1/2)

- Covergroup Basics
- `covergroup` declaration, sampling events (`@`), and options.
- Coverpoints: bins, ranges, wildcards, ignore/illegal bins.
- Cross coverage: when and how to cross (and when not to).
- Sampling Strategy
- Sampling on transactions vs clocks vs protocol events.

2. Functional Coverage Implementation (SV/UVM) (2/2)

- Avoiding oversampling/undersampling.
- Integration with UVM Components
- Embedding covergroups in monitors/scoreboards.
- Using analysis ports/exports for coverage sampling.

3. Code Coverage Setup and Interpretation (1/2)

- Enabling Code Coverage
- Turning on statement/branch/toggle coverage in your simulator.
- Handling compile-time options and run-time switches.
- Interpreting Results
- Reading coverage reports and understanding common metrics.
- Distinguishing real gaps from unreachable or out-of-scope code.

3. Code Coverage Setup and Interpretation (2/2)

- Managing Exclusions
- Identifying truly unreachable code.
- Documenting exclusions and waivers with rationale.

4. Running Coverage-Enabled Regressions (1/2)

- Pilot Regressions
- Start with a modest CORE-tier run (not full regression).
- Ensure coverage data collection is working end-to-end.
- Merging Coverage
- Combining coverage from multiple tests/regressions.
- Understanding how your tool merges coverage across runs and seeds.

4. Running Coverage-Enabled Regressions (2/2)

- Automating Coverage Collection
- Hooks in your regression/CI scripts to generate coverage databases/reports.

5. Coverage Analysis and Gap Closure (1/3)

- Gap Identification
- Finding unhit or low-hit coverpoints and crosses.
- Detecting missing or ineffective coverage (bins that never can be hit).
- Root-Cause Analysis
- Is the gap due to:
- Missing or mis-specified tests?

5. Coverage Analysis and Gap Closure (2/3)

- Overly tight constraints?
- Incomplete coverage model?
- Truly unreachable or out-of-scope functionality?
- Closure Actions
- Adding or modifying tests (directed or constrained-random).
- Adjusting constraints or sequences.

5. Coverage Analysis and Gap Closure (3/3)

- Refining or pruning coverage points.
- Documenting waivers where appropriate.

6. Connecting Coverage to Requirements and Sign-off (1/2)

- Traceability
- Ensuring high-priority requirements are covered by both tests and coverage.
- Updating `REQUIREMENTS_MATRIX.md` with coverage IDs and status.
- Interpreting Coverage for Sign-off
- Understanding when “100% coverage” is meaningful (and when it isn’t).
- Combining coverage with bug metrics and expert judgment.

6. Connecting Coverage to Requirements and Sign-off (2/2)

- Preparing for Later Modules
- How coverage insights will influence later regression and sign-off modules.

Command reference highlights

Scaffold and validate Module 3

- ./scripts/module3.sh --scaffold

Inspect coverage model in TB

- head -60 common_dut/tb/stream_fifo_coverage.sv

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --help
```

Terminal: module3_check

```
[0;36m==========================================[0m
[0;36mModule 3: Coverage Planning & Analysis in Practice[0m
[0;36m==========================================[0m

Usage: ./scripts/module3.sh [OPTIONS]

Module 3: Coverage Planning & Analysis in Practice (SV/UVM)
This script summarizes and checks the coverage artifacts for Module 3.

OPTIONS:
    --design-status       Show status of coverage_design.md only
    --runs-status         Show status of coverage_runs.md only
    --checklist-status    Show status of checklist_module3.md only
    --summary             Show all statuses (default)
    --check               Media/CI self-check (structure only)
    --demo                Print example commands from EXAMPLES.md
    --scaffold            Copy templates/*.md into module3/ if missing
    --help, -h            Show this help message

EXAMPLES:
    # Full summary
    ./scripts/module3.sh

    # Focus only on coverage design
    ./scripts/module3.sh --design-status
```

Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Demo: Coverage design template

```
$ head -30 module3/templates/COVERAGE_DESIGN.md
```

Terminal: ex01_coverage_design

```
# Coverage Design (Module 3)

> **Purpose**: Define the concrete functional coverage model (covergroups/coverpoints/crosses) a
> **Module**: 3 – Coverage Planning & Analysis in Practice (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module3/templates/` if nee

## 1. Context and Scope

<!-- TODO: DUT name, main interfaces, reference docs. Which features/requirements this coverage

## 2. Coverage Components and Placement

<!-- TODO: Table of Coverage ID, UVM Component/File, Sampling Event, Notes. Data source and shar

## 3. Covergroup Specifications

### 3.1 Operations Coverage

<!-- TODO: Covergroup goal, coverpoints, crosses, requirements covered. -->

### 3.2 Occupancy / Level Coverage

<!-- TODO: Covergroup goal, coverpoints, crosses, requirements covered. -->

### 3.3 Backpressure Coverage

<!-- TODO: Covergroup goal, coverpoints, crosses, requirements covered. -->

### 3.4 Error / Status Coverage
```

Demo: Coverage run log template

```
$ head -25 module3/templates/COVERAGE_RUN.md
```

Terminal: ex02_coverage_run

```
# Coverage Runs Log (Module 3)

> **Purpose**: Record coverage-enabled regression runs, key metrics, gaps, and follow-up actions
> **Module**: 3 – Coverage Planning & Analysis in Practice (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module3/templates/` if nee

## 1. Instructions

- Use this file as a **lightweight lab notebook** for coverage work.
- For each significant run, add an entry with:
  - When, what tests, which DUT version, which simulator.
  - Key summary metrics (functional + code).
  - Notable gaps and hypotheses.
  - Planned closure actions.

## 2. Run Entries

### Run 1 – (add title)

<!-- TODO: Date, Engineer, DUT Revision, Environment (simulator, UVM/tool versions). Test Set/Ti

#### 2.1 Functional Coverage Summary (High-Level)

<!-- TODO: Table of Coverage ID, Metric/Notes. -->
```

Demo: TB coverage skeleton

```
$ head -40 common_dut/tb/stream_fifo_coverage.sv
```

Terminal: ex03_tb_coverage

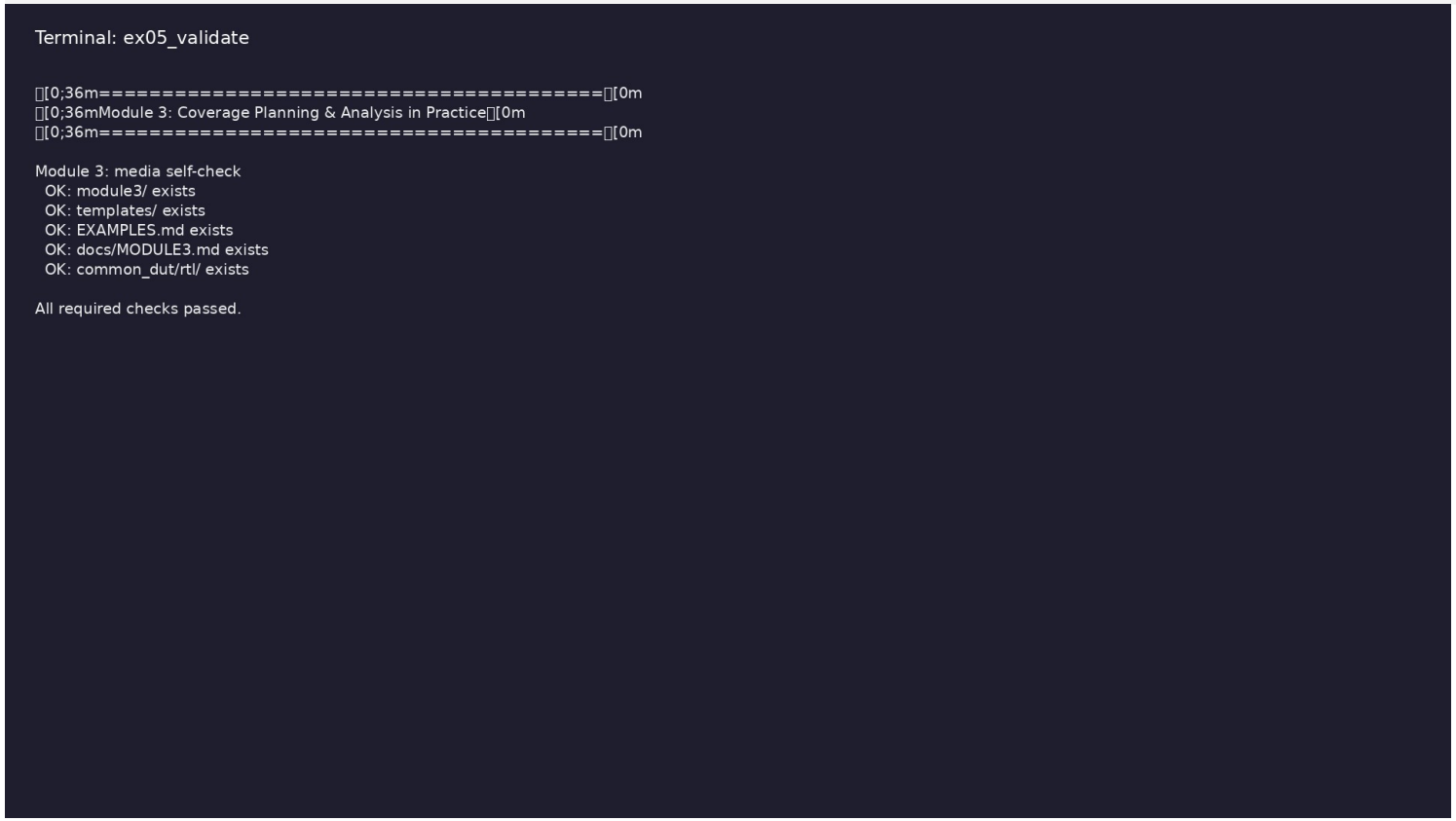
```
// ------------------------------------------------------------------------------
// Functional coverage for stream_fifo (Module 3)
// ------------------------------------------------------------------------------
//
// Covergroups aligned with module3/COVERAGE_DESIGN.md.
// Instantiate this class in stream_fifo_monitor (or equivalent) and call
// sample() on the appropriate events (e.g., posedge clk).
//

`ifndef STREAM_FIFO_COVERAGE_SV
`define STREAM_FIFO_COVERAGE_SV

class stream_fifo_coverage;

  // Inputs to be set by monitor before sample()
  bit [31:0] level;        // Current FIFO occupancy (0..DEPTH)
  bit [1:0]  op_type;      // 0=idle, 1=push, 2=pop, 3=push_and_pop
  bit        overflow;
  bit        underflow;
  int        depth;        // DEPTH parameter for bin bounds

  // ------------------------------------------------------------------------
  // CG_OPS: Operation types per cycle
  // ------------------------------------------------------------------------
  covergroup cg_ops;
    option.per_instance = 1;
    cp_op_type: coverpoint op_type {
      bins idle          = { 0 };
      bins push_only     = { 1 };
      bins pop_only      = { 2 };
      bins push_and_pop  = { 3 };
    }
  endgroup

  // ------------------------------------------------------------------------
  // CG_LEVEL: Occupancy levels (empty / low / mid / high / full)
```

Demo: Reference coverage design

```
$ head -20 module3/.solutions/COVERAGE_DESIGN.md
```

Demo: Module validation

```
$ ./scripts/module3.sh --check
```



Practice & assessment

What you should know (1/5)

- Design and implement a concrete functional coverage model for your DUT.
- Enable and collect code coverage from your simulator.
- Run coverage-enabled regressions and interpret coverage reports.
- Perform a structured coverage gap analysis and plan closure actions.
- Tie coverage results back to requirements and sign-off criteria.
- Choose 2–4 key features or interfaces.

What you should know (2/5)

- For each, implement:
- At least one covergroup with meaningful coverpoints.
- At least one useful cross (if warranted).
- Integrate sampling into your UVM monitor/scoreboard.
- Verify that bins are being hit in basic tests (e.g., SANITY/CORE tests).
- Enable statement/branch coverage in your simulator build/run scripts.

What you should know (3/5)

- Run a small set of tests (e.g., SANITY or a subset of CORE).
- Generate and inspect the coverage report.
- Identify:
- Any obvious coverage gaps in well-understood code.
- Any code that appears to be unreachable or out-of-scope.
- In `module3/COVERAGE_RUN.md`, record at least one coverage run:

What you should know (4/5)

- Date, DUT version, test set/tier, seed policy, simulator version.
- Key coverage metrics (summary) and major observations.
- List at least 5 concrete coverage gaps and your initial hypotheses.
- For at least 3 significant gaps:
- Decide whether to: add tests, tune constraints, refine coverage, or waive.
- Document the chosen action and expected impact.

What you should know (5/5)

- Update:
- `module2/TEST_PLAN.md` (new/updated tests).
- `module2/COVERAGE_PLAN.md` / `module3/COVERAGE_DESIGN.md`.
- `module1/REQUIREMENTS_MATRIX.md` (coverage mappings, waivers).

Exercises

- Over-Coverage vs Under-Coverage
- Coverage vs Bug History (if available)
- Tooling Experiment

Assessment checklist

- Functional coverage model implemented for key features/interfaces.
- Code coverage successfully enabled and reports generated.
- At least one coverage run documented in `COVERAGE_RUN.md`.
- Coverage gaps identified and categorized (test/constraint/model/waiver).
- Closure actions planned and reflected in test/coverage/requirements docs.

Summary & next steps

- Design and implement a practical **coverage model**
  (functional + code), connect it to your UVM env…
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: UVM Environment and Checker Maturity